



A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

Under the guidance of

Dr. Kazi Muheymin-Us-Sakib

Professor, IIT, University of Dhaka

Ratul Sikder | Roll: 2506102 | EMIT

IIT, University of Dhaka

Presentation Outline

01	Motivation & Problem Statement 2025–26 industry losses · Real incidents · Stripe API field changes
02	What is Nexis Core concept · Key guarantees · System overview
03	System Architecture Dual-layer design · Agent roles · Memory & messaging
04	Synthesiser — Challenges & Research Scope LLM synthesis challenges · Hallucination · Multi-provider strategy · Open problems
05	Example Recovery Flow End-to-end walkthrough of a schema drift recovery
06	Technology Stack — Intelligence Claude · OpenAI · Qwen · GLM · Flink · Neo4j
07	Evaluation Plan Metrics · Methods · Baselines

Motivation — The Cost of Unresolved Failures

Quantified industry losses ·

\$9,000

/minute downtime

Avg enterprise IT downtime cost
ITIC Global Survey, 2025

\$10B+

CrowdStrike total

Extended 2024–25 remediation
Parametrix Insurance, 2025

\$2.3B

Azure outage Jan 2025

Global M365 + Azure failure
Microsoft IR + Statista, 2025

\$300B

Annual IT outage cost

Global enterprise losses/yr
ITIC Global Survey, 2025

COMMON INCIDENT

- Silent failure — no automated system detected it before downstream damage spread
- Engineers paged at 2–3 AM, pulled from development into manual incident response
- 2–6 hours of manual log-reading before root cause was even identified
- No tool existed to detect, diagnose, synthesise a fix, and deploy it autonomously
- Monitoring detected symptoms only — none generated or validated a repair

Motivation — Schema & API Field Changes in Production

Real-world Stripe API versioning: December 2024 → April 2026 breaking field renames

REAL-WORLD CASE: STRIPE PAYMENT API

Stripe API — Breaking Field Changes (Dec 2024 – Apr 2026)

2024-12-18 →

`invoice.subscription_details` → `invoice.parent.subscription_details`

2025-01-27 →

`payment_intent.amount_details` → `payment_intent.amount_breakdown (object)`

2025-02-24 →

`charge.billing_details.address` → `charge.billing_details.address_v2`

2025-03-31 →

`payment_intent.status_requires_action` → `payment_intent.next_action.type (enum)`

2025-11-15 →

`subscription.current_period_end` → `subscription.billing_cycle.anchor_end`

2026-02-10 →

`customer.invoice_settings.default_payment_method` → `customer.default_payment_method`

THE ROOT PROBLEM

- Schema and API fields change silently across upstream versions
- No downstream consumer is notified automatically
- Pipelines, dashboards, and ML jobs fail silently for hours
- Engineers discover failures reactively — not proactively
- Manual diagnosis: read logs → trace dependencies → write fix → deploy
- No system exists to detect, diagnose, and repair this automatically
- Every fix requires human involvement at every step

Result: 40–60% of engineer time on maintenance not development (DORA State of DevOps, 2024)

Motivation — Three Failure Modes

Merged problem statement: why engineers are the bottleneck



Hours

before detection

Silent Pipeline Failures

- CI/CD & ETL jobs fail without alerting anyone
- Schema drift propagates undetected downstream
- API contract violations surface as broken dashboards hours later



100%

manual response

The Engineer Bottleneck

- Every incident requires an engineer to be paged
- Manual: log-reading → root-cause → patch → deploy
- Mean time to respond: 81 minutes (Atlassian, 2024)



0

tools close the loop

No Closed-Loop Tool Exists

- Monitoring tools detect — they don't diagnose
- LLM chatbots assist on request — they don't act
- No tool detect → diagnose → repair → validate autonomously

What is Nexis

A team of AI agents that detects, diagnoses, repairs, and validates — autonomously

Nexis models a software engineering team as 9 role-specialised AI agents across two co-ordinated layers. Faults are detected at the moment of occurrence, root-caused via causal inference, repaired by constrained LLM program synthesis, shadow-validated in Docker, and deployed — all without human intervention.

KEY GUARANTEES



Proactive Detection

Fault detected in seconds via Apache Flink streaming and telemetry — not hours later



Causal Root Cause

do-calculus graph traversal pinpoints cause, not just symptoms



Constrained Repair

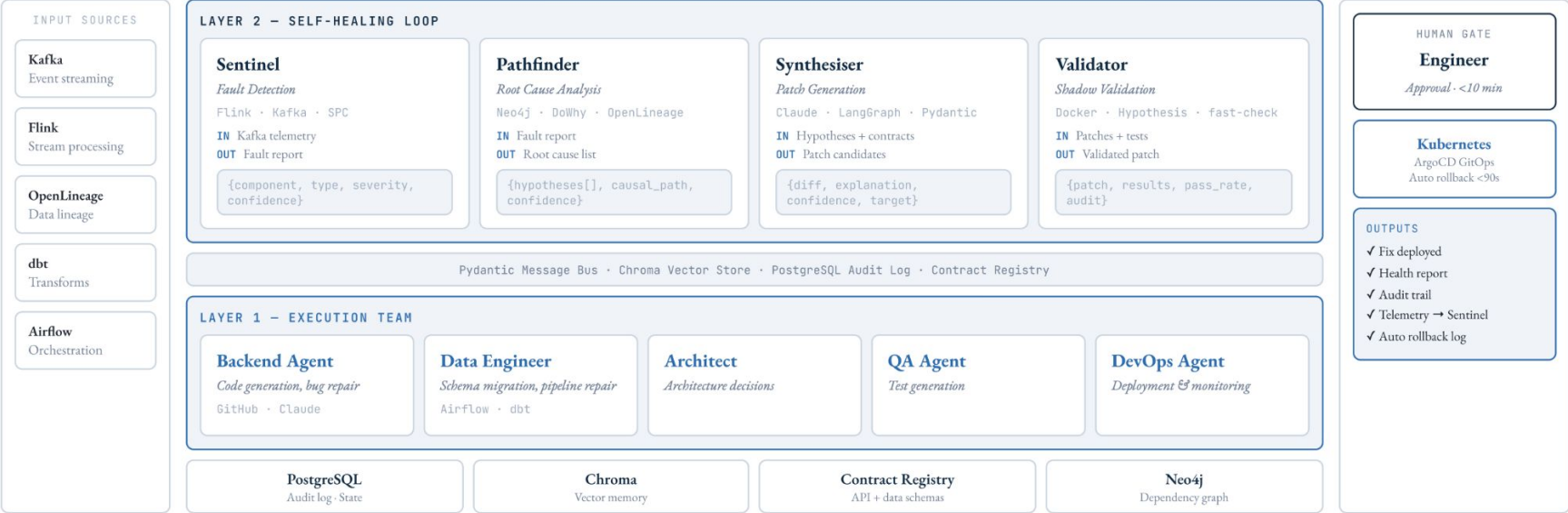
LLM patch synthesis bounded by formal system contracts



Shadow Validation

Zero unvalidated fix ever reaches production — Docker guarantee

System Architecture

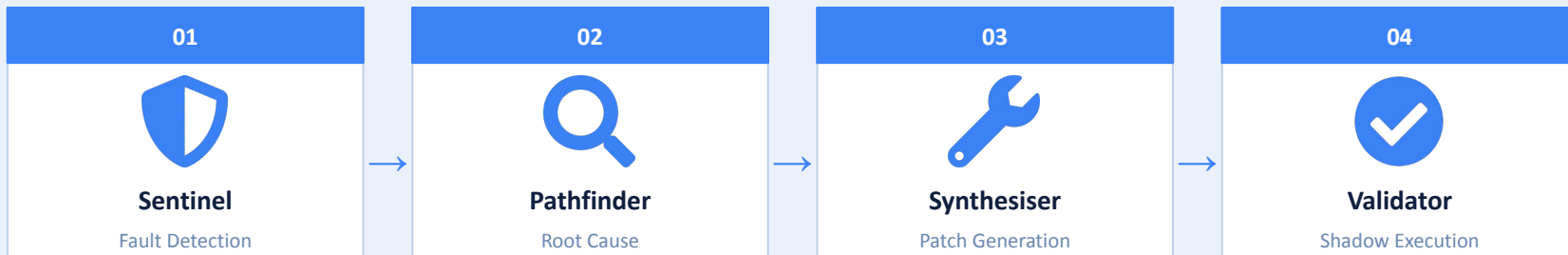


Arrows flow left → right through Layer 2, down through message bus, Layer 1 agents execute delegated tasks, right side shows human gate + deployment

System Architecture — Overview

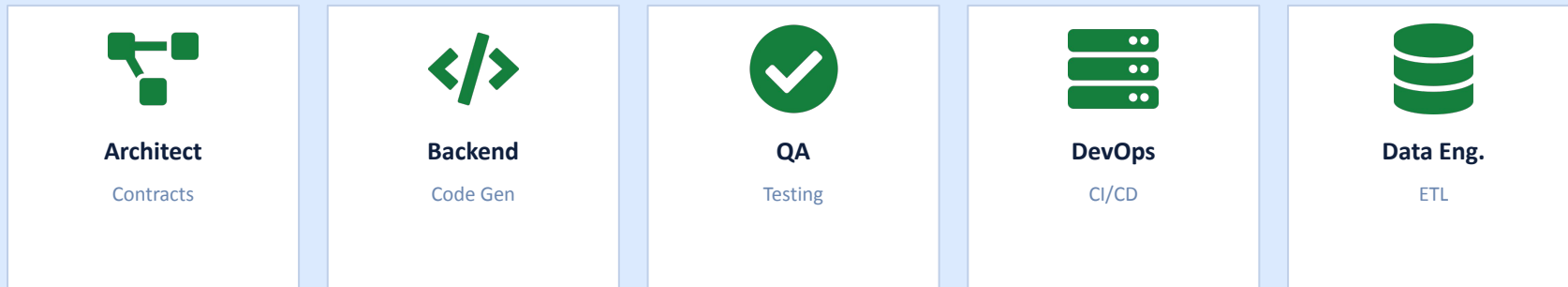
Dual-layer design connected by shared memory and typed messaging

LAYER 2 — SELF-HEALING LOOP (Monitoring & Recovery)



Shared Memory Bus · Chroma (vector) + PostgreSQL (relational) · Pydantic Typed Messaging

LAYER 1 — EXECUTION TEAM (Implementation Agents Pool)



Synthesiser Agent — Challenges & Research Scope

The hardest open problem in Nexis: generating correct, constrained, multi-provider repairs

CORE CHALLENGES

Hallucination in Safety-Critical Paths



- LLMs generate syntactically valid but semantically wrong patches
- A plausible-looking fix that passes shadow tests may still corrupt data
- Mitigation: formal contract constraints + property-based test gating

Context Window Limits on Large Codebases



- Full repo context exceeds token limits of any single LLM
- Selective context injection risks missing critical dependencies
- Research scope: retrieval-augmented synthesis (RAG over codebase)

Multi-Provider Patch Inconsistency



- Claude, GPT-4o, Qwen produce stylistically divergent patches
- Ranking 2–3 candidates from different models is non-trivial
- Research scope: normalised patch scoring + ensemble selection

Patch Ranking Under Uncertainty



- No ground truth available at synthesis time
- Shadow pipeline reduces risk but cannot guarantee production correctness
- Research scope: confidence calibration + RLHF reward modelling

RESEARCH SCOPE & OPEN PROBLEMS



Retrieval-Augmented Synthesis (RAG)

Embed codebase in vector store (Chroma). At repair time, retrieve only the most relevant code context — reducing token usage while preserving dependency awareness.



RLHF Reward Model for Repair Quality

Engineer approval/rejection history trains a reward model. Fine-tunes Qwen2.5-Coder and CodeGLM on domain-specific repair patterns. Patches improve over time.



Multi-LLM Ensemble & Patch Voting

Claude, GPT-4o, Qwen each generate a candidate. A lightweight classifier (trained on shadow test outcomes) selects the best-ranked patch. Reduces single-model failure risk.



Constraint-Guided Decoding

Formal Architect contracts encoded as grammar constraints. Constrained decoding (via Outlines / LMQL) forces LLM output to satisfy schema and type contracts by construction.

Example: End-to-End Schema Drift Recovery

Stripe-style field rename: payment_method_token added as required field in upstream v2.1

Scenario: payment_method_token required field added in payments service v2.1 | 3 downstream consumers (billing, orders, fraud) receive HTTP 500 | 2:14 AM

T + 0s	Sentinel HTTP 500 spike detected across 3 consumers via SPC
T + 31s	Pathfinder Neo4j graph traversal — 14 services examined
T + 58s	Architect Contract violation confirmed and logged
T + 1m 22s	Synthesiser Claude API generates 3 candidate patches
T + 2m 08s	Validator Shadow pipeline: all 3 patches executed in Docker
T + 2m 44s	DevOps Low-risk patch deployed to all 3 consumers autonomously

Technology Stack — Intelligence Layer

LLM-powered synthesis · causal inference · streaming detection · shadow validation

AI / LLM MODELS (MULTI-PROVIDER STRATEGY)

Claude OPUS Anthropic	Primary patch synthesis · plain-English explanations · structured constraint following
GPT-X OpenAI	Fallback synthesis · comparative patch generation · contract interpretation
qwen 3.6 pro Alibaba	Code-specialised generation · efficient fine-tuning for schema repair patterns
GLM-5V-PRO / CodeGLM Zhipu AI	Multi-modal analysis of deployment artefacts · image-based log and dashboard interpretation

OTHER KEY INFRASTRUCTURE

	Apache Flink Sentinel — stateful streaming anomaly detection
	Neo4j + OpenLineage Pathfinder — dependency graph + lineage tracking
	DoWhy / EconML Causal inference — do-calculus root cause ranking
	Docker Shadow Pipeline Validator — isolated parallel execution environment
	Chroma + PostgreSQL Shared memory — vector + relational agent state
	LangGraph / AutoGen Orchestration — typed inter-agent message routing

Evaluation Plan

EVALUATION METHODS

 Fault Injection	 MTTR Measurement	 NASA-TLX User Study	 Patch Correctness
3 fault classes · schema, API, deployment	Auto vs. manual · all fault classes	6 subscales · mental + temporal load	50+ scenarios · shadow-validated

PERFORMANCE TARGETS VS. INDUSTRY BASELINE

Metric	Industry Baseline	Nexis Target	Method
Mean Time to Repair (MTTR)	4–6 hours	< 2 hours (≥60% reduction)	Fault injection + stopwatch
Patch Correctness (first try)	~40%	≥ 80%	Shadow pipeline + prod verify
Engineer effort per incident	2–4 hours	0 min auto / <10 min review	NASA-TLX + time logging
Rollback time	15–30 min	< 90 seconds (automatic)	DevOps Agent timer
Audit trail coverage	Partial / manual	100% automated provenance	Agent log completeness

Research contribution: Nexis demonstrates that a dual-layer multi-agent architecture can achieve reliable end-to-end automation. Primary open challenge: the Synthesiser — generating a correct, safe fix for an unseen failure using only failure evidence and system contracts.

References

- [1] DORA. (2024). State of DevOps Report 2024. Google Cloud / DORA Research Program. dora.dev
- [2] CNCF. (2024). Annual Survey: Cloud Native and Kubernetes Adoption. Cloud Native Computing Foundation.
- [3] Atlassian. (2024). The State of Incident Management 2024. Atlassian Corporation.
- [4] Parametrix Insurance. (2024). CrowdStrike Outage: Economic Impact Analysis. parametrix.ai
- [5] Cyber Risk Alliance. (2022). Log4Shell Global Remediation Cost Report.
- [6] Pearl, J. & Mackenzie, D. (2018). The Book of Why. Basic Books.
- [7] Chen, B., et al. (2023). Automated program repair in the era of large language models. Proc. ICSE 2023.
- [8] Zhang, Y., et al. (2024). Multi-agent systems for autonomous software engineering. ACM Comput. Surv.
- [9] Sharma, A., et al. (2022). Causal inference for root cause analysis in distributed systems. ACM SoCC.
- [10] Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS.
- [11] Stripe. (2024). API Changelog 2011–2024. stripe.com/docs/upgrades
- [12] Gartner. (2023). IT Downtime Cost Research: Critical Infrastructure Metrics. Gartner Research.

THANK YOU